

Уважаемый пользователь!

Обращаем ваше внимание, что система Антиплагиус отвечает на вопрос, является тот или иной фрагмент текста заимствованным или нет. Ответ на вопрос, является ли заимствованный фрагмент именно плагиатом, а не законной цитатой, система оставляет на ваше усмотрение.

Отчет о проверке № 10067864

Дата выгрузки: 2026-06-25 16:27:02

Пользователь: anon_mQqvQenWeB@antiplagius.ru, ID: 10067864

Отчет предоставлен сервисом «Антиплагиат»
на сайте antiplagius.ru/

Информация о документе

№ документа: 10067864

Имя исходного файла: курсач.docx

Размер файла: 0.82 МБ

Размер текста: 39219

Слов в тексте: 5628

Число предложений: 663

Информация об отчете

Дата: 2026-06-25 16:27:02 - Последний готовый отчет

Оценка оригинальности: 100%

Заемствования: 0%

100.00%

0%

Источники:

Доля в тексте

Ссылка

Информация о документе:

Частное учреждение профессионального образования "Высшая школа предпринимательства" (ЧУПО "ВШП") КУРСОВОЙ ПРОЕКТ "Разработка базы данных для системы складского учета" Выполнил: студент 3-го курса специальности 09.02.07 "Информационные системы и программирование" Азизов Сеймур Мубаризович подпись: _____ Проверил: преподаватель дисциплины, преподаватель ЧУПО "ВШП", Честных Всеволод Викторович оценка: _____ подпись: _____

СОДЕРЖАНИЕ ВВЕДЕНИЕ 3 ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОЕКТИРОВАНИЯ БАЗ ДАННЫХ 5 1.1. Анализ предметной области "Складской учёт" 5 1.2. Обзор СУБД и обоснование выбора MySQL 11 1.3. Теоретические основы проектирования и нормализации БД 15 ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ БД 20 2.1. Проектирование структуры БД 20 2.2. Реализация объектов БД в MySQL 26 2.3. Тестирование и примеры использования 31 ЗАКЛЮЧЕНИЕ 35 СПИСОК ИСТОЧНИКОВ 37 ПРИЛОЖЕНИЕ А. Ссылка на репозиторий проекта 39 ПРИЛОЖЕНИЕ Б. Полные листинги SQL-кода 40 ВВЕДЕНИЕ Актуальность темы. В условиях цифровизации экономики эффективное управление товарными запасами становится одним из ключевых факторов конкурентоспособности предприятий торговли и производства. Складское хозяйство является важнейшим звеном логистической цепи: именно здесь концентрируются материальные ценности, происходит их приёмка, размещение, хранение, комплектация и отгрузка. Ошибки в учёте товаров - пересортица, недостатки, излишки, потеря информации о сроках годности - приводят к прямым финансовым потерям, замораживанию оборотных средств и снижению качества обслуживания клиентов [1, с. 12]. Ручной учёт с использованием бумажных журналов или электронных таблиц не обеспечивает целостности и согласованности данных, не позволяет нескольким сотрудникам одновременно работать с информацией и не защищает от противоречивых операций. По данным отраслевых исследований, доля учётных ошибок при ручном ведении складского хозяйства достигает 10-15 %, тогда как внедрение автоматизированной системы на базе СУБД снижает её до 1-2 % [2, с. 34]. Решением данной проблемы является разработка автоматизированной информационной системы, ядром которой выступает реляционная база данных. Именно база данных гарантирует целостность, непротиворечивость и сохранность учётных сведений, а также разграничение прав доступа сотрудников. Таким образом, проектирование и реализация базы данных для системы складского учёта представляет собой актуальную практическую задачу, объединяющую теоретические знания в области проектирования реляционных БД и навыки программирования на языке SQL. Цель работы - спроектировать и реализовать в СУБД MySQL базу данных для системы складского учёта одного склада, удовлетворяющую требованиям целостности, нормализации (не ниже 3НФ) и разграничения прав доступа. Для достижения поставленной цели сформулированы следующие задачи: 1. Проанализировать предметную область "Складской учёт", выделить основные бизнес-процессы, роли

участников и сущности. 2. Провести сравнительный обзор современных реляционных СУБД и обосновать выбор MySQL для реализации проекта. 3. Изучить теоретические основы проектирования и нормализации баз данных, рассмотреть нормальные формы и принципы ER-моделирования. 4. Разработать концептуальную и логическую модели базы данных, построить ER-диаграмму и привести структуру к третьей нормальной форме. 5. Реализовать физическую структуру БД в MySQL средствами языка DDL, создать таблицы, роли и назначить права доступа. 6. Реализовать программные объекты БД: хранимую процедуру, представление, триггер, пользовательскую функцию, транзакцию с обработкой исключений. 7. Провести тестирование базы данных на наборе типовых запросов и подготовить проектную документацию. Объект исследования - процессы складского учёта товарно-материальных ценностей на предприятии. Предмет исследования - методы и средства проектирования и реализации реляционной базы данных для автоматизации складского учёта в СУБД MySQL. Методы исследования: анализ предметной области и научной литературы, методы инфологического (ER) и даталогического моделирования, метод нормализации отношений, метод декомпозиции, экспериментальное тестирование разработанной базы данных. Структура работы. работа состоит из введения, двух глав, заключения, списка источников и приложений. В первой главе рассматриваются теоретические основы предметной области и проектирования БД. Во второй главе представлены практическое проектирование, реализация и тестирование базы данных. Практическая значимость работы заключается в том, что разработанная база данных может быть использована в качестве основы для построения полнофункциональной информационной системы складского учёта малого или среднего предприятия, а также как учебный пример для изучения дисциплины. Все исходные материалы размещены в публичном репозитории и пригодны для дальнейшего развития. ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОЕКТИРОВАНИЯ БАЗ ДАННЫХ 1.1. Анализ предметной области "Складской учёт" Складской учёт - это система регистрации и контроля движения товарно-материальных ценностей (ТМЦ) на складе предприятия. Его основная задача - обеспечить точную и оперативную информацию о наличии, поступлении и расходе товаров, их размещении и состоянии [1, с. 45]. Качественно организованный складской учёт позволяет минимизировать издержки хранения, ускорить обработку заказов и предотвратить как избыточное накопление запасов, так и их дефицит. Перед проектированием базы данных необходимо детально описать предметную область: бизнес-процессы, участников (роли) и основные сущности. Бизнес-процессы складского учёта Складская деятельность представляет собой последовательность взаимосвязанных операций, образующих жизненный цикл товара на складе - от поступления до отгрузки. Рассмотрим каждый процесс подробно. 1. Приёмка товара. Это первый этап жизненного цикла товара на складе. При поступлении партии от поставщика кладовщик сверяет фактическое количество и номенклатуру с сопроводительными документами (товарной накладной, счётно-фактурой). Проверяется целостность упаковок, при необходимости фиксируются номер партии и срок годности. По результатам приёмки в системе формируется приходная операция (тип IN), которая увеличивает остаток соответствующего товара. Корректность приёмки критична, так как ошибки на этом этапе распространяются на все последующие операции и искажают учётные данные. 2. Размещение. После приёмки товар физически размещается в одной из адресуемых зон (ячеек) хранения. Каждая зона имеет уникальный адрес, кодируемый по принципу "ряд-стеллаж-полка" (например, А-01-03). Адресное хранение существенно ускоряет последующий поиск и комплектацию, а также позволяет учитывать особенности товара (например, размещать крупногабаритные позиции в специально отведённой зоне). В базе данных привязка товара к зоне отражается в таблице остатков. 3. Хранение. На этапе хранения товар находится в зоне до момента его отбора. Здесь важны контроль сроков годности, поддержание условий хранения (температурный режим, влажность) и постоянный мониторинг текущих остатков. Учётная система должна в любой момент предоставлять достоверную информацию о количестве товара в каждой зоне. 4. Комплектация (подбор). При поступлении заказа на отгрузку выполняется комплектация - подбор требуемых позиций товара в нужном количестве из мест хранения. Эффективная комплектация опирается на адресную систему хранения и сведения об остатках, что позволяет построить оптимальный маршрут отбора и сократить время сборки заказа. 5. Отгрузка. Скомплектованный заказ передаётся получателю - клиенту или другому подразделению. В системе формируется расходная операция (тип OUT), уменьшающая остаток товара. Отгрузка завершает основной цикл движения товара и фиксируется в журнале операций с указанием исполнителя и времени. 6. Инвентаризация. Периодически проводится сверка фактического наличия товаров с учётными данными. По результатам инвентаризации корректируются остатки, выявляются недостачи (фактическое количество меньше учётного) и излишки (больше учётного). Инвентаризация обеспечивает достоверность учёта и является обязательной процедурой материальной ответственности. 7. Перемещение. Внутрискладское перемещение - это перенос товара между зонами хранения без изменения суммарного остатка по складу. Например, товар может быть перемещён из зоны приёмки в зону основного хранения. Поскольку операция затрагивает сразу две записи остатков (списание из исходной зоны и зачисление в целевую), она должна выполняться как единая транзакция: либо обе записи изменяются согласованно, либо не изменяется ни одна. Это предотвращает рассогласование данных при сбоях. Рисунок 1.1 - Схема основных бизнес-процессов склада Как видно из рисунка 1.1, процессы приёмки, размещения, хранения, комплектации и отгрузки образуют линейный поток движения товара, а инвентаризация и перемещение являются вспомогательными процессами, воздействующими на этап хранения. Все процессы оставляют "след" в журнале операций, что обеспечивает прослеживаемость движения каждой позиции. Участники и роли В системе складского учёта взаимодействуют сотрудники с различными полномочиями. Для обеспечения безопасности данных и разграничения ответственности выделены две основные роли. Администратор (admin) - сотрудник, отвечающий за настройку и сопровождение информационной системы. Имеет полный доступ ко всем данным и объектам базы данных, управляет справочниками (категории, поставщики, зоны), учётными записями пользователей и правами доступа. Администратор может создавать и изменять структуру БД, выполнять резервное копирование и восстановление данных. Кладовщик (storekeeper) - материально ответственный сотрудник, выполняющий повседневные складские операции: приёмку, размещение, комплектацию, отгрузку и перемещение товаров. Имеет ограниченный доступ - может просматривать справочники и добавлять операции, изменять остатки, но не вправе изменять структуру БД, удалять данные или управлять учётными

записями. Такое разграничение реализует принцип минимальных привилегий и снижает риск случайной или преднамеренной порчи данных. ДействиеАдминистраторКладовщикПросмотр справочников и остатковДаДаРегистрация складских операцийДаДаИзменение справочников (товары, поставщики)ДаЧастичноУдаление данныхДаНетИзменение структуры таблицДаНетУправление пользователями и правамиДаНетТаблица 1.1 - Матрица прав доступа ролей Основные сущности предметной области На основе анализа бизнес-процессов и участников выделены ключевые сущности, информация о которых должна храниться в базе данных (таблица 1.2). СущностьНазначениеКатегория (category)Группировка товаров по типам для удобства поиска и отчетностиТовар (product)Номенклатура товаров - основной справочникПоставщик (supplier)Контрагенты, поставляющие товары на складЗона (zone)Адресуемые места хранения внутри складаОстатки (stock)Количество конкретного товара в конкретной зонеОперация (operation)Зарегистрированный факт движения товара (приход, расход, перемещение)Таблица 1.2 - Основные сущности предметной области Дополнительно выделяется сущность "Пользователь" (app_user), хранящая сведения о сотрудниках-исполнителях операций. Данный набор сущностей и связей между ними станет основой для построения ER-модели (глава 2).

1.2. Обзор СУБД и обоснование выбора MySQL Система управления базами данных (СУБД) - это комплекс программных средств, обеспечивающих создание, ведение и совместное многопользовательское использование баз данных [3, с. 22]. Современный рынок предлагает множество реляционных СУБД, различающихся лицензионной политикой, производительностью, набором функций и сложностью администрирования. Для обоснованного выбора платформы реализации проекта рассмотрены четыре наиболее распространённые реляционные СУБД. MySQL - свободно распространяемая реляционная СУБД, разрабатываемая корпорацией Oracle. Отличается высокой производительностью на операциях чтения, простотой развёртывания и обширным сообществом. При использовании движка InnoDB поддерживает транзакции (ACID), внешние ключи, хранимые процедуры, триггеры и представления. Широко применяется в веб-разработке (стек LAMP/LEMP). PostgreSQL - объектно-реляционная СУБД с открытым исходным кодом, известная строгим соблюдением стандарта SQL, поддержкой расширенных типов данных (JSON, массивы, геоданные, пользовательские типы), а также развитыми механизмами для сложной аналитики. Считается более "академически правильной", но требует несколько большего опыта администрирования. Microsoft SQL Server - коммерческая СУБД корпорации Microsoft, тесно интегрированная с экосистемой Windows и платформой .NET. Обладает мощными средствами бизнес-аналитики (SSAS, SSRS) и удобными инструментами администрирования (SSMS), однако полнофункциональные редакции являются платными. Oracle Database - флагманская коммерческая СУБД корпоративного уровня, обеспечивающая максимальную функциональность, надёжность и масштабируемость. Применяется в крупных банковских и государственных системах. Отличается высокой стоимостью лицензий и сложностью администрирования, что делает её избыточной для небольших проектов. КритерийMySQLPostgreSQLMS SQLOracleЛицензияGPL / свободнаяOpen SourceКоммерч.Коммерч.СтоимостьБесплатноБесплатноВысокаяОчень высокаяПорог входаНизкийСреднийСреднийВысокийТранзакции (ACID)Да (InnoDB)ДаДаДаПроцедуры, триггерыДаДаДаДаКроссплатформенностьДаДаОгранич.ДаСообществоОчень большоеБольшоеБольшоеБольшоеТаблица 1.3 - Общее сравнение реляционных СУБД Для более детального обоснования в таблице 1.4 приведена дополнительная техническая характеристика рассматриваемых СУБД по параметрам, существенным для учебного проекта складского учёта.

ПараметрMySQLPostgreSQLMS SQLOracleПоддержка Unicode (utf8mb4)ПолнаяПолнаяПолнаяПолнаяИнструменты GUIWorkbenchpgAdminSSMSSQL DeveloperОбъём установочного пакетаМалый (~300 МБ)СреднийБольшойОчень большойПростота резервного копированияВысокаяВысокаяСредняяСредняяОбъём обучающих материаловМаксимальныйБольшойБольшойБольшойТребования к ресурсамНизкиеСредниеВысокиеОчень высокиеТаблица 1.4 - Дополнительные технические характеристики СУБД Обоснование выбора MySQL. Для данного учебного проекта масштаба одного склада выбрана СУБД MySQL Community Edition. Аргументы в пользу выбора проекта приведены ниже [4, с. 18]. 1. Бесплатность и открытость - отсутствие лицензионных отчислений, что критично для учебного и малобюджетного проекта; редакция Community Edition полностью функциональна. 2. Низкий порог вхождения - простота установки и наличие удобных графических инструментов (MySQL Workbench, phpMyAdmin), а также обширная база обучающих материалов на русском языке. 3. Достаточная функциональность - движок InnoDB обеспечивает поддержку транзакций (ACID), внешних ключей, триггеров, хранимых процедур, функций и представлений - всех объектов, требуемых заданием. 4. Корректная работа с кириллицей - кодировка utf8mb4 обеспечивает полную поддержку Unicode, включая хранение русскоязычных наименований товаров и сотрудников без искажений. 5. Низкие требования к ресурсам - СУБД стабильно работает даже на маломощном оборудовании, что удобно для разработки и демонстрации. 6. Широкая распространённость - MySQL является промышленным стандартом веб-разработки, поэтому приобретаемые навыки имеют высокую практическую ценность на рынке труда. Таким образом, для проекта масштаба одного склада возможностей MySQL более чем достаточно, а простота сопровождения и бесплатность делают её оптимальным выбором среди рассмотренных альтернатив.

1.3. Теоретические основы проектирования и нормализации БД ER-моделирование Проектирование базы данных принято разделять на три этапа: концептуальное (инфологическое), логическое (дatalogическое) и физическое. На концептуальном этапе строится модель "сущность-связь" (Entity-Relationship, ER), предложенная П. Ченом в 1976 году [5, с. 30]. Эта модель описывает предметную область в терминах, понятных как разработчику, так и заказчику, не привязываясь к конкретной СУБД. Основными понятиями ER-модели являются: 1. сущность (entity) - объект предметной области, о котором необходимо хранить информацию (например, "Товар", "Поставщик"); 2. атрибут (attribute) - свойство сущности (например, наименование, цена, единица измерения); 3. связь (relationship) - логическая ассоциация между сущностями (например, товар "поставляется" поставщиком); 4. ключ (key) - атрибут или набор атрибутов, однозначно идентифицирующий каждый экземпляр сущности. Типы связей Между сущностями выделяют три основных типа связей, определяющих кардинальность ассоциации [5, с. 41]. "Один-к-одному" (1:1) - одному экземпляру первой сущности соответствует не более одного экземпляра второй, и наоборот. Например, "сотрудник - учётная запись". "Один-ко-многим" (1:N) - одному экземпляру первой сущности может соответствовать множество экземпляров второй, но

каждому экземпляру второй - только один первой. Это наиболее распространённый тип. Пример из проекта: один поставщик поставляет множество товаров, но каждый товар закреплён за одним поставщиком. "Многие-ко-многим" (M:N) - множеству экземпляров первой сущности соответствует множество экземпляров второй. В реляционной модели такая связь не реализуется напрямую и разбивается на две связи 1:N через промежуточную (связующую) таблицу. В проекте связь "товар - зона" реализована через таблицу остатков stock: один товар может храниться в нескольких зонах, а в одной зоне - несколько товаров.

Нормальные формы Нормализация - это процесс пошагового приведения структуры базы данных к виду, минимизирующему избыточность данных и устраняющему три типа аномалий: вставки, обновления и удаления [2, с. 78]. Нормализация выполняется путём декомпозиции отношений с сохранением информации. Рассмотрим нормальные формы с примерами применительно к складскому учёту. Первая нормальная форма (1НФ). отношение находится в 1НФ, если все его атрибуты атомарны (неделимы), отсутствуют повторяющиеся группы и каждая ячейка содержит единственное значение. Например, недопустимо хранить в одном поле "контакты" поставщика и телефон, и e-mail через запятую - это нарушает атомарность. Корректное решение - разнести их по отдельным полям phone и email, что и сделано в проекте. Вторая нормальная форма (2НФ). отношение находится в 2НФ, если оно в 1НФ и каждый неключевой атрибут функционально полно зависит от всего первичного ключа, а не от его части. 2НФ актуальна лишь для отношений с составным ключом. Пример нарушения: если бы таблица операций имела составной ключ (товар, дата) и одновременно хранила наименование товара, то наименование зависело бы только от товара - части ключа. В проекте все таблицы используют простой суррогатный ключ, поэтому 2НФ соблюдается автоматически. Третья нормальная форма (3НФ). отношение в 3НФ, если оно в 2НФ и отсутствуют транзитивные зависимости - то есть ни один неключевой атрибут не зависит от другого неключевого атрибута. Пример нарушения: если в таблице product хранить и category_id, и наименование категории category_name, то возникнет транзитивная зависимость $product_id \rightarrow category_id \rightarrow category_name$. Это приведёт к избыточности (название категории дублируется у каждого товара) и аномалиям обновления. Решение - вынести категорию в отдельную таблицу category и хранить в product только внешний ключ category_id, что и реализовано. Нормальная форма Бойса-Кодда (BCNF). усиленная 3НФ: для любой нетриви